

Design and Implementation of a Computerized Task Scheduling System

By

Gamaliel Egbuchiri

AFIT/ICT/18/0014

Supervised By

Mr. Ifeanyi Frederick Okonkwo

Submitted to

Information and Communication Technology Department

Faculty of Ground and Communication Engineering

Air Force Institute of Technology, Kaduna

In Partial Fulfilment of the Requirements for the Award of Bachelor of Engineering

(B.ENG.) Degree in Information and Communication Technology

September, 2024

DECLARATION

I, Gamaliel Egbuchiri, with Matric Number AFIT/ICT/18/0014, hereby declare that this project work titled Design and Implementation of a Computerized Task Scheduling System is a record of an original work done by me under the supervision of Mr. Ifeanyi Frederick Okonkwo, in the Department of Information and Communication Technology, Faculty of Ground and Communication Engineering, Air Force Institute of Technology, Kaduna.

_____	_____	_____
<i>Name of Student</i>	<i>Signature</i>	<i>Date</i>

CERTIFICATION

This is to certify that this study titled Design and Implementation of a Computerized Task Scheduling System was carried out by Gamaliel Egbuchiri with Matric Number AFIT/ICT/18/0014 in the Department of Information and Communication Technology, Faculty of Ground and Communication Engineering, Air Force Institute of Technology, Kaduna, Nigeria.

_____	_____	_____
<i>Supervisor</i>	<i>Sign & Date</i>	

_____	_____	_____
<i>Head of Department</i>	<i>Sign & Date</i>	

_____	_____	_____
<i>External Examiner</i>	<i>Sign & Date</i>	

DEDICATION

This work is dedicated with profound gratitude to God Almighty, whose grace, wisdom, and provision sustained me through every stage of this project. It is also dedicated to my parents, whose unwavering love, sacrifice, and encouragement have been the bedrock of every achievement in my life.

ACKNOWLEDGEMENT

I give all glory to God Almighty for the knowledge, strength, and perseverance to complete this project. My heartfelt appreciation goes to my supervisor, Mr. Ifeanyi Frederick Okonkwo, whose consistent guidance, constructive feedback, and encouragement were invaluable throughout the development of this work. I also acknowledge the project coordinator and all members of the project committee for their support and direction, which ensured the timely completion of this work. My sincere thanks go to all the lecturers in the Department of Information and Communication Technology at the Air Force Institute of Technology, Kaduna, for the knowledge they imparted over the course of my study. I appreciate my colleagues and friends who assisted in testing the system, providing feedback, and offering encouragement at various stages of development. Finally, I am grateful to my family for their all-round support, which made this journey both bearable and rewarding.

ABSTRACT

Modern organizations and individuals operate in environments characterized by increasing complexity, tight deadlines, and competing priorities. Manual task management approaches — whether paper-based or relying on generic tools such as spreadsheets — frequently lead to missed deadlines, poor workload visibility, and inefficient resource allocation. Recognizing this challenge, this project presents the design and implementation of Task Flow, a computerized task scheduling system built entirely with client-side web technologies: HyperText Markup Language (HTML), Cascading Style Sheets (CSS), and JavaScript. TaskFlow provides a comprehensive suite of features including task creation, editing, rescheduling, prioritisation (Critical, High, Medium, Low), soft-deletion with trash recovery, real-time search, sorting, category management, progress tracking with visual progress bars, subtask management with automatic progress computation, starred tasks, repeat scheduling, analytics dashboard, and a monthly calendar view. The system operates without any server-side infrastructure, making it deployable directly from a web browser, and persists all user data using the browser's built-in localStorage API. The Big Bang Software Development Life Cycle (SDLC) model was adopted as the development methodology, which suited the scope of the project and allowed for flexible, iterative development. Testing was conducted on a modern desktop browser environment and confirmed that all stated objectives were met. The system produces a responsive, modern interface with a dark-sidebar design, colour-coded priority indicators, and real-time feedback through toast notifications. This project contributes to a practical, accessible tool to the domain of computerized task scheduling, with recommendations for future work including backend integration, multi-user collaboration, and mobile application development.

TABLE OF CONTENTS

DECLARATION	i
CERTIFICATION	ii
DEDICATION	iii
ACKNOWLEDGEMENT	iv
ABSTRACT	v
TABLE OF CONTENTS	vi
LIST OF FIGURES AND TABLES	ix
CHAPTER ONE: INTRODUCTION	1
1.1 Background to the Study	1
1.2 Statement of the Problem	3
1.3 Aim and Objectives of the Study	4
1.4 Scope of the Study	4
1.5 Justification of the Study	5
1.6 Outline of Thesis	5
CHAPTER TWO: LITERATURE REVIEW	6
2.1 Introduction	6
2.2 Fundamental Concepts	6
2.2.1 Task Scheduling and Management	6

2.2.2 Web-Based Applications	8
2.2.3 Frontend Web Technologies	9
2.2.4 Data Persistence in Browser Applications	11
2.2.5 Existing Task Management Systems	12
2.2.6 Software Development Methodologies	14
2.3 Related Works	16
2.3.1 Summary of Related Works	18
 CHAPTER THREE: METHODOLOGY	20
3.1 Introduction	20
3.2 Workflow	20
3.3 Requirement Analysis	22
3.3.1 Data Acquisition	22
3.3.2 Software and Hardware Requirements	22
3.4 Use Case Analysis	24
3.5 System Design	26
3.5.1 Architecture Design	26
3.5.2 Data Model	27
3.5.3 System Flowchart	28
3.5.4 Activity Flow	29
3.6 Implementation	30
3.6.1 Frontend Structure (HTML)	30
3.6.2 Styling (CSS)	31
3.6.3 Application Logic (JavaScript)	32

CHAPTER FOUR: RESULTS AND DISCUSSION 34

4.1 Introduction	34
4.2 Main User Interface	34
4.3 Task Creation	36
4.4 Trash and Recovery	37
4.5 Search and Filtering	38
4.6 Analytics Dashboard	39

CHAPTER FIVE: SUMMARY AND CONCLUSION 41

5.1 Summary	41
5.2 Conclusion	42
5.3 Recommendation	43

REFERENCES 45

LIST OF FIGURES AND TABLES

Figures

Figure 2.1: TaskFlow Technology Stack	9
Figure 2.2: Big Bang SDLC Model Applied to TaskFlow	15
Figure 3.1: The Workflow Diagram	21
Figure 3.2: TaskFlow Use Case Diagram	25
Figure 3.3: System Architecture Diagram	26
Figure 3.4: TaskFlow Data Model	27
Figure 3.5: System Flowchart	28
Figure 3.6: User Activity Flow Diagram	29
Figure 3.7: HTML Source Code (index.html) in VS Code	30
Figure 3.8: CSS Source Code (style.css) in VS Code	31
Figure 3.9: JavaScript Source Code (app.js) in VS Code	32
Figure 4.1: TaskFlow Main User Interface — All Tasks View	35
Figure 4.2: New Task Modal Dialog	36
Figure 4.3: Trash View Showing Deleted Tasks	37
Figure 4.4: Permanent Delete Confirmation Dialog	38
Figure 4.5: Real-Time Search Filtering Tasks	39
Figure 4.6: Analytics Dashboard	40

Tables

Table 2.1: Comparison of Web Technologies Used in TaskFlow	10
Table 2.2: Summary of Related Works	18
Table 3.1: TaskFlow Feature Summary	23

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

In the contemporary world, effective management of tasks and schedules has become a fundamental requirement for both individual productivity and organizational efficiency. Whether in an academic setting, a corporate environment, or personal life, the ability to plan, prioritize, track, and complete tasks in a systematic manner is directly linked to the achievement of goals and deadlines. As digital transformation continues to reshape how individuals and institutions operate, there is a growing demand for software tools that can replace fragmented, manual scheduling methods with integrated, intelligent systems.

Task scheduling, in the context of information systems, refers to the organized allocation of tasks to time slots, priorities, and responsible parties, with mechanisms for monitoring progress and ensuring timely completion. Traditional approaches to task management — including paper-based to-do lists, physical noticeboards, and simple spreadsheet applications — have significant limitations. They offer little support for priority visualization, real-time progress tracking, or dynamic rescheduling. Furthermore, they provide no mechanism for recovering deleted items, searching across entries, or generating aggregate statistics about productivity patterns.

The proliferation of web technologies has made it possible to develop sophisticated, interactive applications that run entirely within the web browser, without requiring installation or server infrastructure. The three foundational technologies of the modern web — HTML for structure, CSS for presentation, and JavaScript for behavior — have matured to a point where feature-rich applications previously possible only in native desktop environments can now be delivered as lightweight, cross-platform web pages (Flanagan, 2020). Coupled with browser-side storage APIs such as local Storage, these technologies enable the creation of fully functional, offline-capable task management systems.

This project responds to the need for a modern, accessible, and feature-complete task scheduling system by designing and implementing Task Flow — a computerized task scheduling system developed entirely in vanilla HTML, CSS, and JavaScript. Task Flow is designed to offer a professional-grade task management experience, including task creation and editing, prioritization, progress tracking, subtask management, rescheduling, soft-deletion with trash recovery, category organization, starred tasks, analytics, and calendar views.

The interface of Task Flow draws on contemporary design principles, featuring a dark-themed sidebar combined with a clean light content area, color-coded priority badges, and smooth visual transitions. These design choices are grounded in usability research demonstrating that consistent visual hierarchies and immediate feedback mechanisms significantly improve user task-completion rates (Norman, 2013). The system stores all data locally in the browser, making it usable offline without any account creation or internet dependency.

1.2 Statement of the Problem

Despite the wide availability of digital devices and the internet, many students, professionals, and organizations continue to rely on informal or inadequate methods of task tracking. Spreadsheets are used as to-do lists but offer no built-in priority system or progress indicators. Physical notepads lack searchability and cannot generate reports. General-purpose tools such as Microsoft Outlook or Google Calendar focus primarily on calendar events rather than task workflows.

Existing commercial task management applications such as Todoist, Asana, and Trello, while feature-rich, present several challenges. They require constant internet connectivity, impose subscription fees for advanced features, present complex interfaces that discourage adoption among less technical users, and do not operate in a fully offline, standalone manner (Markus & Mao, 2004). There is therefore a clear need for a task scheduling system that operates independently of any server infrastructure, is built on open and universally supported web technologies, provides a clean and modern interface suitable for non-technical users, and incorporates all core task management features without external dependencies.

1.3 Aim and Objectives of the Study

The aim of this project is to design and implement a computerized web-based task scheduling system that enables users to manage tasks effectively through a modern, accessible browser interface.

The specific objectives are:

- To review and analyze existing task management systems and web-based application development approaches.

- To design a user-friendly, responsive web interface for task scheduling and management.
- To implement the full range of task management features including creation, editing, prioritization, rescheduling, search, filtering, and analytics.
- To implement persistent local data storage using the browser's local Storage API.
- To evaluate the implemented system against its stated requirements through functional testing.

1.4 Scope of the Study

This project is scoped to the design and implementation of a client-side, browser-based task scheduling system built exclusively using HTML, CSS, and JavaScript — without any back-end server, database management system, or external framework. Data persistence is achieved through the browser's local Storage API. The system supports individual user operation on a single device. Multi-user collaboration, cloud synchronization, mobile native applications, and server-side features are outside the scope of this project and are recommended as directions for future development. The system was tested on Google Chrome (version 126) running on a Windows 11 desktop environment.

1.5 Justification of the Study

The development of Task Flow is justified by the genuine productivity needs of students and professionals who require a reliable, feature-rich task management tool that does not depend on subscription services, internet connectivity, or installation procedures. By delivering a fully functional scheduling system as a set of three static files (HTML, CSS, JavaScript), this project demonstrates that high-quality productivity software can be built and distributed at zero cost

using open web standards. Furthermore, the project contributes pedagogical value by illustrating best practices in vanilla JavaScript application development, including event delegation, local Storage integration, modular function design, and dynamic DOM manipulation — all without the overhead of a JavaScript framework.

1.6 Outline of Thesis

This project report is organized into five chapters. Chapter One provides the background, problem statement, aim, objectives, scope, and justification of the study. Chapter Two reviews the fundamental concepts, related literature, and existing systems relevant to the project. Chapter Three describes the methodology, system design including case analysis, architecture, data model, and implementation details. Chapter Four presents the results and a discussion of the system's features and performance. Chapter Five concludes the report with a summary, conclusions, and recommendations for future work.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

This chapter provides a comprehensive review of the concepts, technologies, and related works that underpin the development of the Task Flow system. It begins with an exploration of the fundamental concepts of task scheduling and management, followed by an examination of web-based application development using client-side technologies. The chapter then reviews existing task management systems and their limitations and concludes with a summary of related works and the research gaps that this project addresses.

2.2 Fundamental Concepts

2.2.1 Task Scheduling and Management

Task scheduling is the process of assigning tasks to time periods, resources, or sequences in a manner that optimizes some objective — typically the timely completion of all tasks while respecting constraints such as priorities, deadlines, and dependencies (Pinedo, 2016). In the context of personal productivity and project management, a task represents a discrete unit of work that has a defined title, optional description, due date, priority level, and completion status.

Effective task management encompasses several interrelated activities: planning (defining what needs to be done), prioritization (ranking tasks by importance or urgency), scheduling (assigning tasks to time slots), tracking (monitoring progress), and review (evaluating

outcomes). Researchers in the field of Human-Computer Interaction have identified that digital task management tools must provide adequate support for all these activities to be genuinely useful (Bernstein et al., 2014). Priority management is particularly critical. The Eisenhower Matrix, popularized by Covey (2004) in the 7 Habits of Highly Effective People, classifies tasks along two axes — urgency and importance — mapping naturally onto the four-level priority system (Critical, High, Medium, Low) implemented in Task Flow.

2.2.2 Web-Based Applications

A web application is a software application that runs in a web browser, accessed through a network such as the internet or an intranet. Unlike traditional desktop applications, web applications do not require installation and are platform-independent, running on any device with a compatible browser (Flanagan, 2020). Web applications are broadly categorized into server-side applications, which process logic on a remote server and return HTML to the client, and client-side applications, which execute all logic within the browser using JavaScript.

Client-side web applications — also known as Single-Page Applications (SPAs) when they manage navigation without full page reloads — have become increasingly powerful with the maturation of JavaScript and the availability of browser APIs such as local Storage, IndexedDB, and the Fetch API. Task Flow falls into this category: it is a client-side application that performs all computation within the browser and stores data using the local Storage API, requiring no server whatsoever. The absence of a server component offers significant advantages in terms of deployment simplicity and accessibility — a Task Flow user can open the application by simply opening the index.html file in any modern browser with no installation, no sign-up, and no internet connection required after the initial page load.

2.2.3 Frontend Web Technologies

The three foundational technologies of client-side web development are HTML, CSS, and JavaScript. HTML5 provides the semantic structure of a web page, introducing a rich vocabulary of semantic elements that improve accessibility and usability (W3C, 2014). CSS3 controls the visual presentation, providing layout systems including Flexbox and CSS Grid, as well as custom properties (variables), transitions, and animations that enable sophisticated responsive interfaces without JavaScript (MDN Web Docs, 2023). JavaScript is the programming language of the web, providing dynamic behavior through DOM manipulation, event handling, and interaction with browser APIs. Modern JavaScript (ES6+) provides class syntax, arrow functions, destructuring, template literals, and modules (ECMA International, 2023). Figure 2.1 below illustrates the technology stack used in Task Flow.

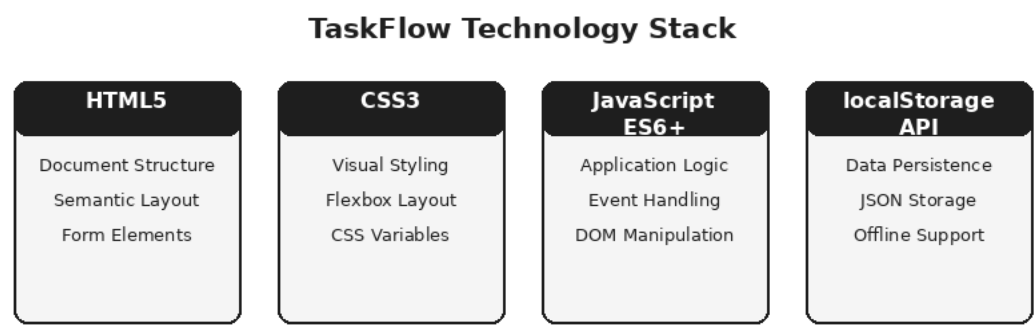


Figure 2.1: TaskFlow Technology Stack

Table 2.1 below presents a comparison of the three web technologies as applied in the Task Flow system.

Table 2.1: Comparison of Web Technologies Used in TaskFlow

Feature	HTML	CSS	JavaScript	Overall Role
Type	Markup	Stylesheet	Script	Structure / Style / Logic
Server Needed	No	No	No	All client-side
Data Storage	N/A	N/A	localStorage	JS handles all storage
Learning Curve	Low	Low	Moderate	Beginner-friendly stack

2.2.4 Data Persistence in Browser Applications

Data persistence in client-side web applications — the ability for data to survive page refreshes or browser restarts — is achieved through browser storage APIs. The local Storage API provides synchronous key-value storage scoped to the current origin. It can store up to approximately 5 MB of data per origin and persists indefinitely until explicitly cleared. Data is stored as strings, requiring JSON serialization and deserialization for structured data (Frisbie, 2019). Task Flow uses local Storage as its persistence layer, serializing the tasks and trash arrays as JSON strings on every state-changing operation and deserialising them on application startup. IndexedDB, by contrast, provides an asynchronous, transactional database within the browser capable of storing larger volumes of structured data. While more powerful, it introduces significant API complexity. For the scale of data managed by Task Flow (an individual user's tasks), local Storage is entirely adequate and preferable for its simplicity.

2.2.5 Existing Task Management Systems

The landscape of digital task management tools is broad. Todoist (Doist, 2023) is a cloud-based task management application offering prioritization, recurring tasks, labels, and productivity analytics — but it requires internet connectivity and restricts advanced features to paid subscribers. Microsoft To Do (Microsoft, 2023) provides task lists, due dates, and reminders but requires a Microsoft account and constant internet connectivity. Trello (Atlassian, 2023) uses the Kanban board metaphor and is designed for team workflows rather than individual task scheduling, with a column-based interface that does not naturally support priority-based ordering. Notion (Notion Labs, 2023) is an all-in-one workspace combining notes, databases, and tasks, but has a steep learning curve and is cloud dependent. In contrast, Task Flow operates entirely offline, requires no account creation, imposes no subscription fees, and presents a focused interface designed specifically for individual task scheduling without external dependencies.

2.2.6 Software Development Methodologies

The Software Development Life Cycle (SDLC) is a structured process used to plan, create, test, and deliver software systems. The Waterfall Model is a sequential phase-by-phase approach suited to well-understood projects, but inflexible in the face of changing requirements (Royce, 1970). The Agile Model employs iterative development in short cycles, continuously incorporating feedback and well-suited to projects with evolving requirements (Beck et al., 2001). The Big Bang Model is an approach in which development proceeds with minimal upfront planning, driven primarily by available resources and implementation knowledge, best suited to small, well-scoped projects undertaken by individual developers (Tutorialspoint, 2023). For this project — a single-developer, well-defined web application — the Big Bang Model was selected. Figure 2.2 illustrates the Big Bang SDLC model as applied to this project.

Big Bang SDLC Model — Applied to TaskFlow

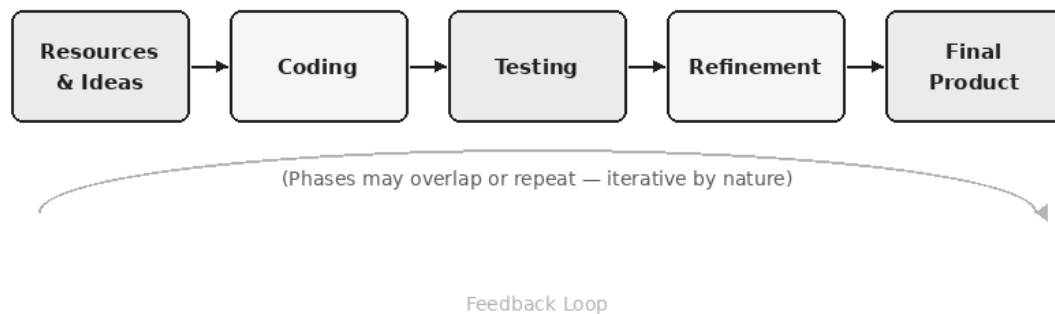


Figure 2.2: Big Bang SDLC Model Applied to TaskFlow

2.3 Related Works

Several studies and systems in the literature relate closely to the present work.

Bernstein et al. (2014) conducted a study on the design of personal task management tools and identified several key usability requirements: rapid task capture, easy prioritization, progress visibility, and recovery from deletion. Their findings informed the feature set of Task Flow, particularly the progress bar, priority badges, and trash recovery system.

Ahmed et al. (2020) developed a web-based project management system using Node.js and MongoDB, demonstrating the feasibility of browser-based task management. Their system, however, required server infrastructure and user registration — limitations that Task Flow addresses.

Salami and Okon (2021) reviewed existing digital task management tools used in Nigerian universities and found significant dissatisfaction with complexity, internet dependency, and cost. They recommended the development of lightweight, offline-capable tools tailored to resource-constrained contexts — a recommendation that directly motivates the present project.

Khalid et al. (2020) developed a web-based scheduling tool using PHP and MySQL, emphasizing the importance of priority-based filtering and deadline notifications. Their user interface design insights — particularly the use of color coding for priorities — were influential in the design of Task Flow's priority badge system.

Okafor and Eze (2022) examined the use of local Storage for offline web application development, demonstrating that browser-native storage is sufficient for single-user productivity applications and that JSON serialization introduces negligible performance overhead for typical task volumes. These findings support the technical approach adopted in Task Flow.

Flanagan (2020) in *JavaScript: The Definitive Guide*, provides a comprehensive treatment of DOM manipulation, event handling, and browser APIs including local Storage. This reference informed the implementation patterns used throughout the Task Flow application logic.

2.3.1 Summary of Related Works

Table 2.2: Summary of Related Works

Author & Year	Title / Focus	Method	Limitation
Bernstein et al. (2014)	Task management usability requirements	User study, design analysis	No system implementation; theoretical only
Ahmed et al. (2020)	Web-based project management system	Node.js, MongoDB backend	Requires server and internet connectivity
Salami & Okon (2021)	Digital task tools in Nigerian universities	Survey and review	No practical implementation provided
Khalid et al. (2020)	Priority-based web scheduling tool	PHP, MySQL server-side	Requires server, no offline support
Okafor & Eze (2022)	localStorage for offline web apps	Performance benchmarking	Limited to storage benchmarks only
Flanagan (2020)	JavaScript: The Definitive Guide	Technical reference	Reference text, not an implemented system

CHAPTER THREE

METHODOLOGY

3.1 Introduction

This chapter details the methodology adopted for the design and implementation of the Task Flow computerized task scheduling system. It covers the workflow of the development process, the requirement analysis phase, use case analysis, system design including architecture, data model, flowchart and activity diagram, and the implementation details of each technology component.

3.2 Workflow

The Big Bang SDLC model was selected as the development methodology for this project. Under this model, the development process is driven by available resources and a clear understanding of requirements, proceeding through implementation with continuous refinement. The workflow of the project is illustrated in Figure 3.1 below.

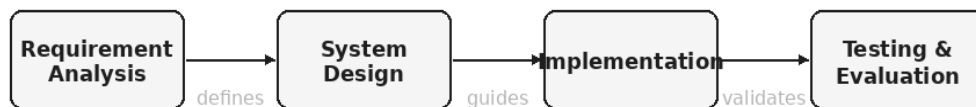


Figure 3.1: The Workflow Diagram

Each phase of the workflow feeds into the next. Requirement analysis informs system design; design guides implementation; and implementation outputs are evaluated through functional testing. The Big Bang model's flexibility allowed for revisiting earlier phases as implementation insights emerged, making it particularly suitable for a single-developer project of this nature.

3.3 Requirement Analysis

3.3.1 Data Acquisition

Information requirements for the Task Flow system were gathered through three primary sources. First, a review of related literature and existing task management systems (as presented in Chapter Two) identified the key features that users expect from a modern task scheduling tool. Second, an analysis of the limitations of existing systems — particularly their dependency on internet connectivity, server infrastructure, and subscription payments — informed the architectural decisions of the project. Third, informal consultation with students and staff at the Air Force Institute of Technology provided practical feedback on desired features such as priority categorization, a trash recovery mechanism, and a progress indicator.

3.3.2 Software and Hardware Requirements

Task Flow is a client-side web application and therefore has minimal software and hardware requirements. Software requirements include: (i) a modern web browser (Google Chrome 110+, Mozilla Firefox 110+, or equivalent) supporting ES6+ JavaScript and the local Storage API; and (ii) a text editor or Integrated Development Environment — Visual Studio Code was used for this project. No server software, database, or runtime environment is required for

deployment. Hardware requirements are minimal: any device capable of running a modern web browser is sufficient. The development device used had the following specifications:

- Processor: Intel Core i5-10th generation, 2.4 GHz
- RAM: 8 GB
- Operating System: Windows 11 Home
- Browser: Google Chrome 126

Table 3.1 below presents a summary of the features implemented in the Task Flow system.

Table 3.1: TaskFlow Feature Summary

Feature	Description
Task Management	Add, edit, duplicate, delete tasks with full metadata
Priority Labels	Critical, High, Medium, Low — colour-coded badges
Progress Bar	Per-task progress (0–100%) with dynamic colour fill
Subtasks	Checklists that automatically update parent progress
Trash & Restore	Soft-delete with restore or permanent removal
Analytics Dashboard	Stats, priority breakdown, category charts, effort distribution
Calendar View	Monthly calendar showing task due-date distribution
Real-Time Search	Searches titles, notes, tags across all tasks instantly
Sorting & Filtering	Sort by 5 criteria; filter chips by priority/status

Data Persistence	Tasks saved to localStorage, survive page refresh
Keyboard Shortcuts	Ctrl+N (new task), Ctrl+K (search), Esc (close)
Repeat Schedule	Daily, weekly, monthly, or one-time task scheduling

3.4 Use Case Analysis

A use case analysis was conducted to identify the interactions between the system and its primary actor — the User. A use case represents a specific goal that the actor wants to accomplish using the system (Jacobson et al., 1999). Since Task Flow is a single-user application, all use cases are associated with one actor. The use case diagram, presented in Figure 3.2, identifies sixteen distinct use cases that the system must support.

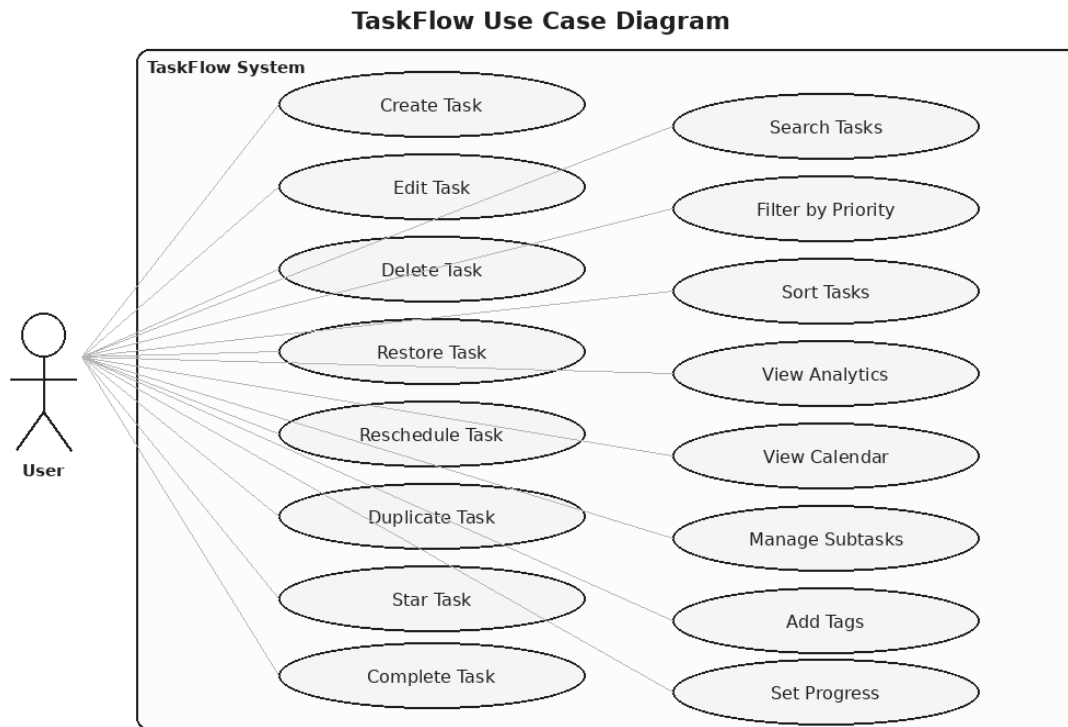


Figure 3.2: TaskFlow Use Case Diagram

The use cases are grouped into two categories. Task Operation use cases cover the core task lifecycle: creating, editing, deleting, restoring, rescheduling, duplicating, starring, and completing tasks. These operations represent the primary interactions a user has with individual tasks. View and Organization use cases cover higher-level interactions: searching tasks, filtering by priority, sorting the task list, viewing the analytics dashboard, viewing the calendar, managing subtasks within a task, adding tags to tasks, and setting task progress manually. Each use case is described below:

1. Create Task — The user opens the New Task modal, fills in the required title and optional fields (priority, category, due date, effort, subtasks, tags, progress), and saves. The system validates the title and adds the task to the top of the list, saving to local Storage.
2. Edit Task — The user clicks the edit icon on a task card. The modal opens pre-populated with the task's current data. The user modifies any field and saves. The system updates the task in-place and re-renders.
3. Delete Task — The user clicks the trash icon. The task is moved to the trash array (soft-delete), removed from the active task list, and local Storage is updated. The trash badge counts increments.
4. Restore Task — In the Trash view, the user clicks Restore. The task is moved back to the active task list, and local Storage is updated.

5. Reschedule Task — The user clicks the calendar icon on a task card. The modal opens with an additional Reschedule Reason field. On saving, the task's due date is updated, and a rescheduled badge is displayed on the card.
6. Duplicate Task — The user clicks the copy icon. A new task is created with the same data, a new unique ID, zero progress, and "Copy of" prepended to the title.
7. Star Task — The user clicks the star icon. The task starred flag is toggled; starred tasks always appear at the top of the list regardless of the current sort order.
8. Complete Task — The user clicks the circular checkbox. The task's done flag is toggled and progress is set to 100%. The card is visually dimmed and the title is struck through.
9. Search Tasks — The user types in the search box. The task list filters in real time to show only tasks matching the query in title, notes, category, or tags.
10. Filter by Priority — The user clicks a priority filter chip. The task list immediately shows only tasks of the selected priority level.
11. Sort Tasks — The user selects a sort criterion from the dropdown (due date, priority, created, progress, or name). The list re-orders immediately.
12. View Analytics — The user navigates to the Analytics view. A dashboard with summary statistics, priority and category bar charts, overall progress, and effort distribution is displayed.
13. View Calendar — The user navigates to the Calendar view. A monthly grid shows days with due tasks highlighted, and an upcoming deadlines list is shown below.
14. Manage Subtasks — Within the task modal, the user adds subtasks and checks them off. Checking a subtask updates the parent task's progress proportionally.

15. Add Tags — The user enters comma-separated tags in the tags field of the task modal.

Tags are displayed as small badges on the task card and are included in search matching.

16. Set Progress — The user drags the progress range slider in the task modal. The percentage display updates in real time, and the task's progress bar reflects the new value on save.

3.5 System Design

3.5.1 Architecture Design

The architecture of Task Flow is a purely client-side architecture, as illustrated in Figure 3.3. Unlike traditional three-tier web applications (presentation, application logic, data), Task Flow collapses all three tiers into a single client-side layer executing within the browser. The three source files — `index.html`, `style.css`, and `app.js` — are loaded by the browser. The JavaScript module manages all application state, handles user interactions through event listeners, renders the UI by generating and inserting HTML into the DOM, and reads/writes to local Storage on every state change.

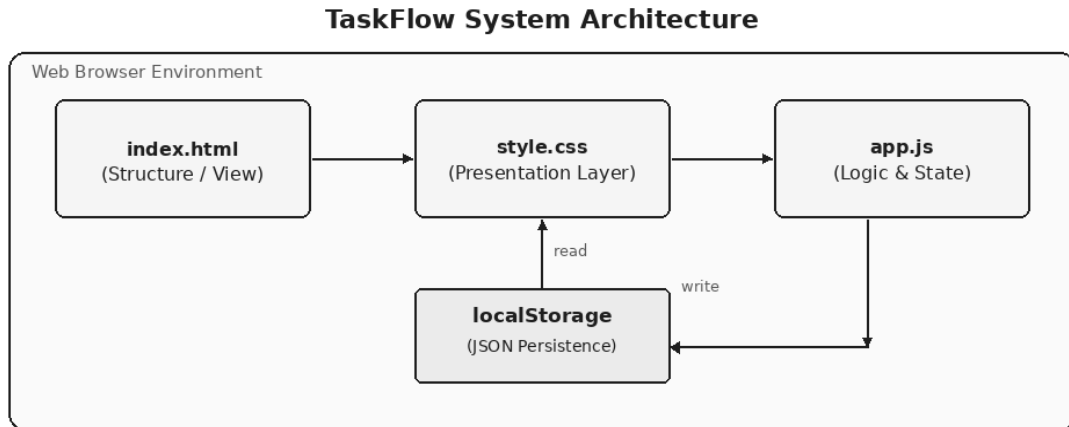


Figure 3.3: System Architecture Diagram

3.5.2 Data Model

The data model of Task Flow is based on two arrays stored in local Storage: the tasks array (active tasks) and the trash array (soft-deleted tasks). Each element in both arrays is a Task object with the fields shown in Figure 3.4 below. The Task object has a one-to-many relationship with Subtask objects, which are stored as an embedded array within each Task. This embedded document pattern avoids the need for relational joins and is well-suited to the key-value storage model of local Storage.

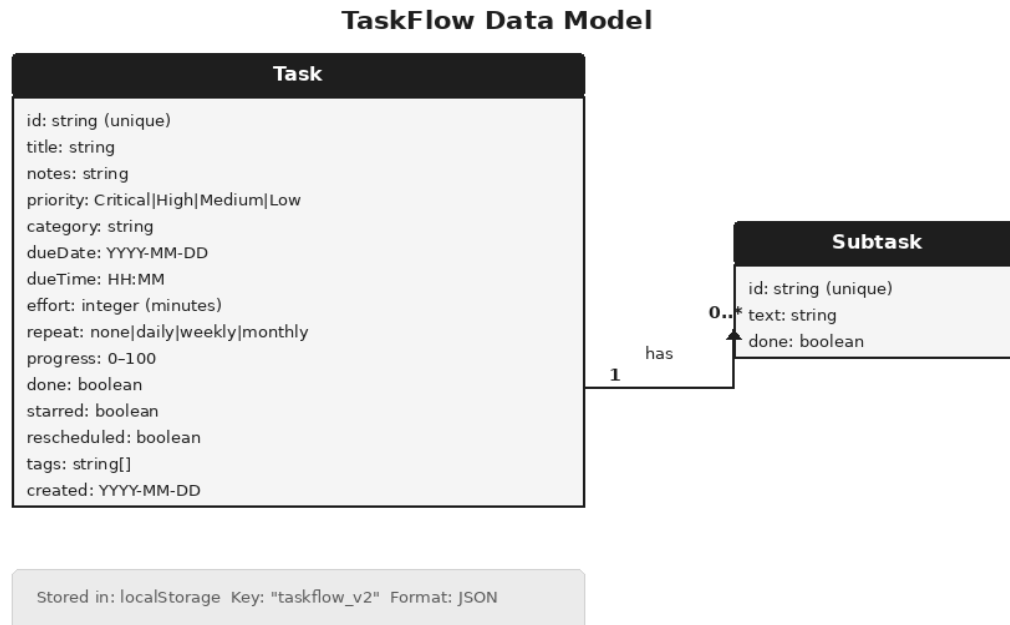


Figure 3.4: TaskFlow Data Model

3.5.3 System Flowchart

Figure 3.5 illustrates the decision logic of the Task Flow system from startup through user interaction and state persistence. On startup, the system checks local Storage for existing data; if none is found, seed data is loaded. After rendering the task list, the system waits for user interaction. On interaction, the appropriate action is executed. Destructive actions (permanent delete, empty trash) trigger a confirmation dialog before proceeding. After every action, the state is saved to local Storage and the interface is re-rendered. A feedback loop returns the system to the waiting state.

TaskFlow System Flowchart

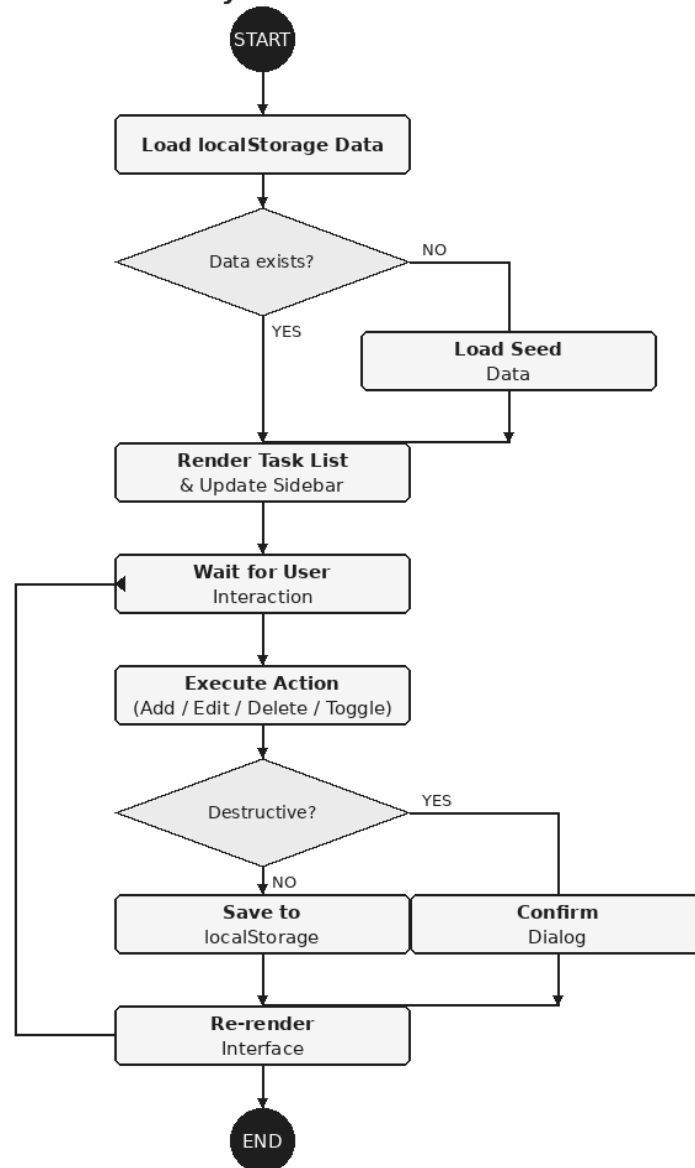


Figure 3.5: System Flowchart

3.5.4 Activity Flow

Figure 3.6 illustrates the user activity flow, showing the main sequence of interactions a user follows from opening the application through to saving changes. The diagram highlights three

common sub-activities branching from the "Choose Action" step: adding a new task, editing or completing an existing task, and deleting a task to the trash.

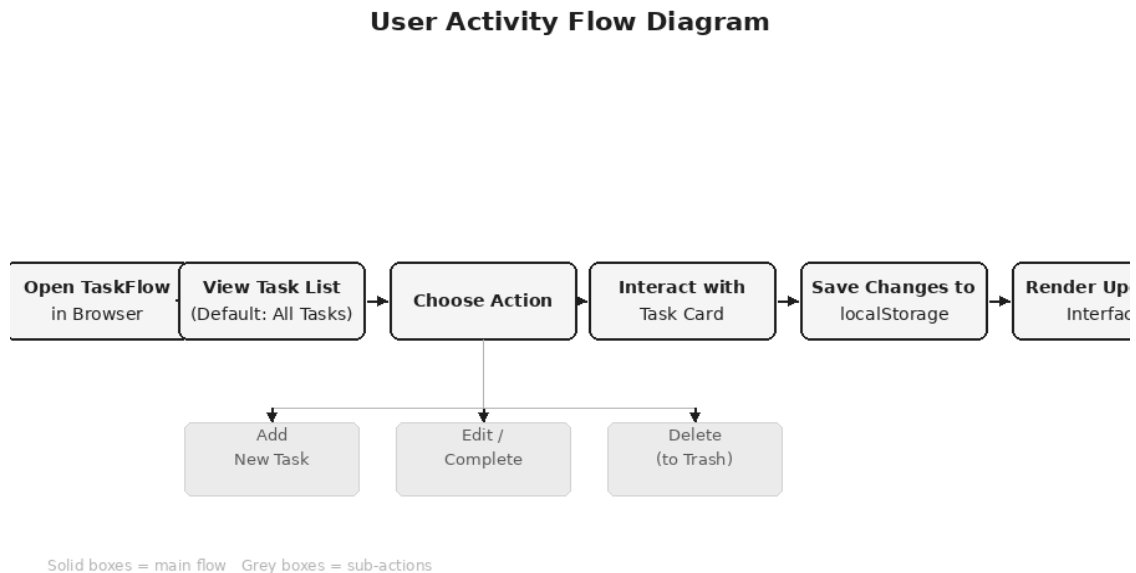


Figure 3.6: User Activity Flow Diagram

3.6 Implementation

3.6.1 Frontend Structure (HTML)

The index.html file defines the complete semantic structure of the Task Flow application. It consists of: (i) a sidebar element containing the application logo, navigation links with live badge counts, a dynamic category navigation section, and a statistics panel; (ii) a main element containing the topbar (with title, search box, sort selector, and new task button) and the content area (with filter chips and the task list container); (iii) a modal overlay for the new/edit task form; (iv) a confirmation dialog overlay for destructive actions; and (v) a toast notification container. The HTML source code is shown in Figure 3.7 below.

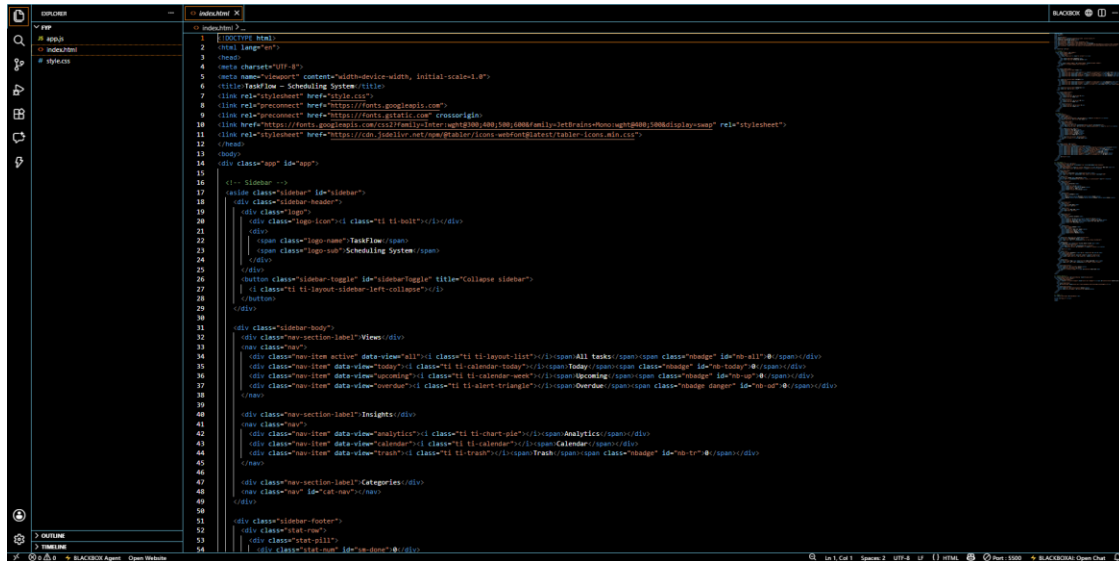


Figure 3.7: HTML Source Code (index.html) in VS Code

3.6.2 Styling (CSS)

The style.css file contains all visual styling for the TaskFlow application. The design system is built on CSS custom properties (variables) defined in the: root selector, establishing the colour palette, typography scale, spacing system, border radii, and shadow tokens. The primary colour scheme uses a deep navy sidebar background contrasted with a light content area, with an indigo accent for interactive elements. Priority colours follow a semantic mapping: coral-red for Critical, orange for High, indigo for Medium, and teal for Low. The layout uses CSS Flexbox for the two-column app layout. The CSS source code is shown in Figure 3.8.

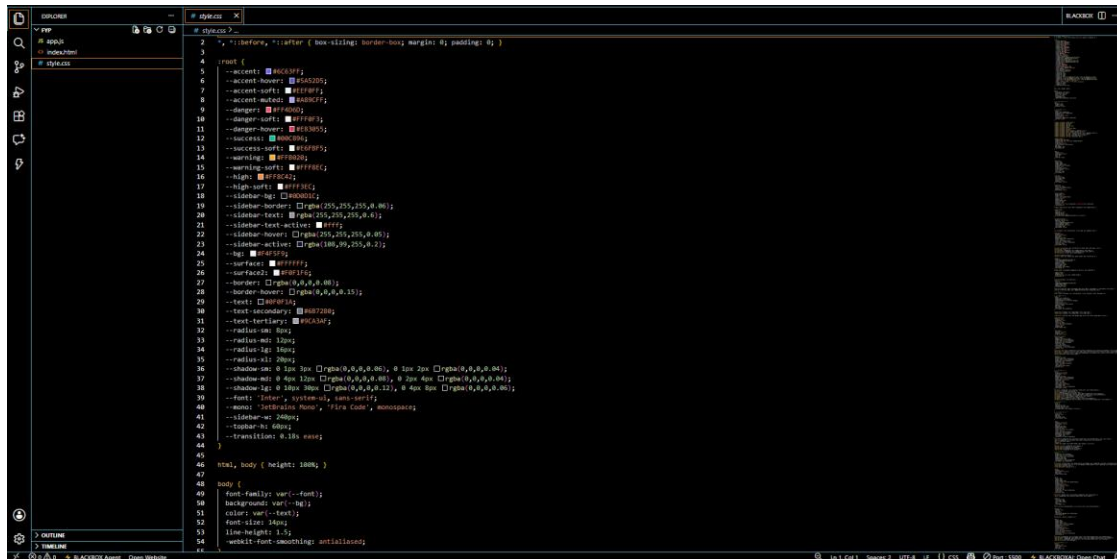


Figure 3.8: CSS Source Code (style.css) in VS Code

3.6.3 Application Logic (JavaScript)

The app.js file contains all application logic, organised into the following sections: (i) state variables (tasks, trash, editId, view, filter, mSubs, confirmCallback, catFilter); (ii) storage functions — save() serialises state to localStorage, load() deserialises on startup; (iii) seed data — demonstration tasks loaded on first run; (iv) filtering and sorting — getFiltered() applies view, category, search, and filter constraints, then sorts; (v) render functions — render() dispatches to renderAnalytics(), renderCalendar(), or renderTrash() based on the current view; (vi) modal functions for opening, closing, and resetting the task form; (vii) action functions for all CRUD and interaction operations; and (viii) event listeners wired in a DOMContentLoaded callback using event delegation. The JavaScript source code is shown in Figure 3.9.



Figure 3.9: JavaScript Source Code (app.js) in VS Code

A key implementation pattern is event delegation: rather than attaching individual event listeners to each task card button, a single listener is attached to the task list container and uses `event.target.closest("[data-action]")` to identify the action and its associated task ID. This pattern significantly improves performance for large task lists and avoids memory leaks from event listener accumulation. The `localStorage` integration uses a simple serialisation/deserialisation pattern: `save()` calls `JSON.stringify` on the state object and writes it to a single key (`taskflow_v2`), while `load()` reads and parses this key on startup. The version suffix in the key name provides a forward-migration path should the data schema change in a future version.

CHAPTER FOUR

RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents the results of the implementation and testing of the TaskFlow computerised task scheduling system. It demonstrates the system's key features through annotated screenshots taken from the operational system running in Google Chrome. The chapter covers the main user interface, task creation, trash and recovery, search and filtering, and the analytics dashboard. Each section relates the observed behaviour of the system to the objectives stated in Chapter One and the use cases identified in Chapter Three.

4.2 Main User Interface

Figure 4.1 shows the main TaskFlow interface in the "All Tasks" view. The interface is divided into a dark sidebar on the left and a light content area on the right. The sidebar displays navigation links with live task counts shown as badges, a category section that dynamically builds entries from the user's task categories, and a statistics panel at the bottom showing total completed tasks, completion rate, and total task count.

The main area shows the topbar with the view title, a real-time search box, a sort selector, and the prominent "New task" button. Below the topbar, filter chips allow rapid filtering by priority level or completion status. Task cards are displayed in the task list, each showing the task title, priority badge, category badge, due date with effort estimate, and a colour-coded progress bar.

Hovering over a task card reveals action buttons for editing, rescheduling, duplicating, and deleting the task.

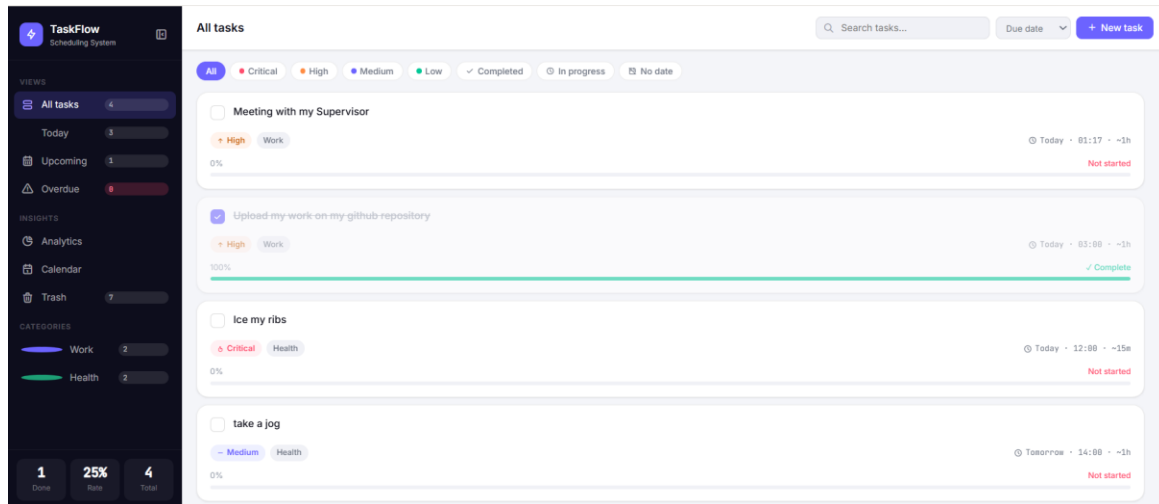


Figure 4.1: TaskFlow Main User Interface — All Tasks View

The interface achieves the design objective of a modern, professional appearance. Priority badges are rendered in distinct colours (red for Critical, orange for High, indigo for Medium, teal for Low), providing immediate visual differentiation without requiring the user to read the label text. The progress bar uses a dynamic colour scale from red (0%) through amber (30%) to indigo (60%) to teal (100%), giving an at-a-glance indication of task completion status. Overdue tasks receive a red left border accent, immediately drawing the user's attention to items requiring urgent action. The sidebar navigation updates its badge counts in real time after every action, ensuring the user always has an accurate count of tasks in each view.

4.3 Task Creation

Figure 4.2 shows the New Task modal dialog, which opens when the user clicks the "New task" button or presses Ctrl+N. The modal contains fields for task title (required, validated on submission), notes/description, priority selector, category selector, due date, due time, effort estimate, repeat schedule, subtask input with an add button, a progress range slider with live percentage display, and a tags input.

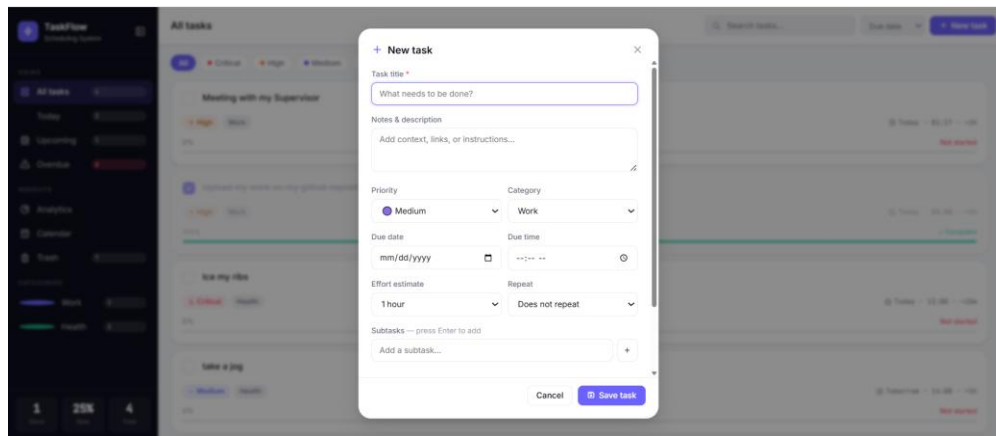


Figure 4.2: New Task Modal Dialog

The modal demonstrates several usability features. The title field receives focus automatically on modal open, allowing immediate typing. Pressing Enter in the title field submits the form. Pressing Enter in the subtask field adds the subtask and clears the input, enabling rapid subtask entry. The progress slider updates its numeric display in real time as it is dragged. The modal can be dismissed by clicking outside it or pressing Escape. When the form is submitted without a title, the title field's border turns red and a toast notification appears: "Task title is required." On successful submission, the new task appears at the top of the task list and the sidebar

statistics update immediately. The rescheduling function opens the same modal with an additional Reschedule Reason field; if a reason is entered, the task is marked with a rescheduled badge providing a clear audit trail.

4.4 Trash and Recovery

Figure 4.3 shows the Trash view, accessible via the Trash navigation item in the sidebar. When a task is deleted by clicking the trash icon on its card, it is moved to the trash array rather than immediately destroyed. This soft-deletion approach mirrors the behaviour of modern operating systems and significantly reduces the risk of unrecoverable data loss.

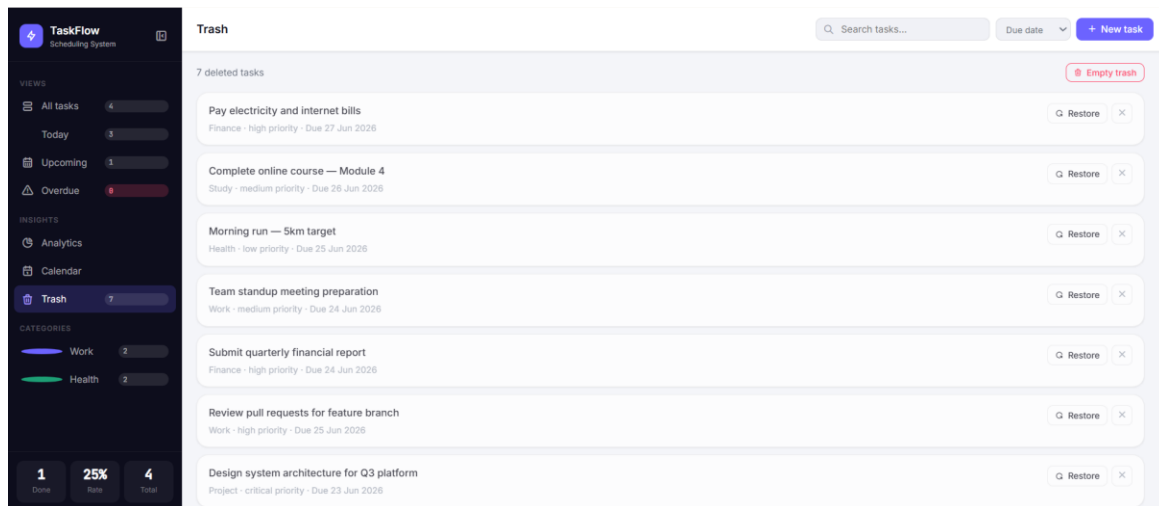


Figure 4.3: Trash View Showing Deleted Tasks

The Trash view displays all deleted tasks with their category, priority, and due date shown as metadata. Each trashed task has a Restore button that returns it to the active task list, and an "X" button that initiates permanent deletion. An Empty Trash button at the top right permanently removes all trashed tasks. Figure 4.4 shows the confirmation dialog that appears

when the user attempts to permanently delete a task or empty the trash. The dialog presents a clear warning message and requires explicit confirmation before any irreversible action is taken. This two-step confirmation pattern is a standard usability practice for destructive operations.

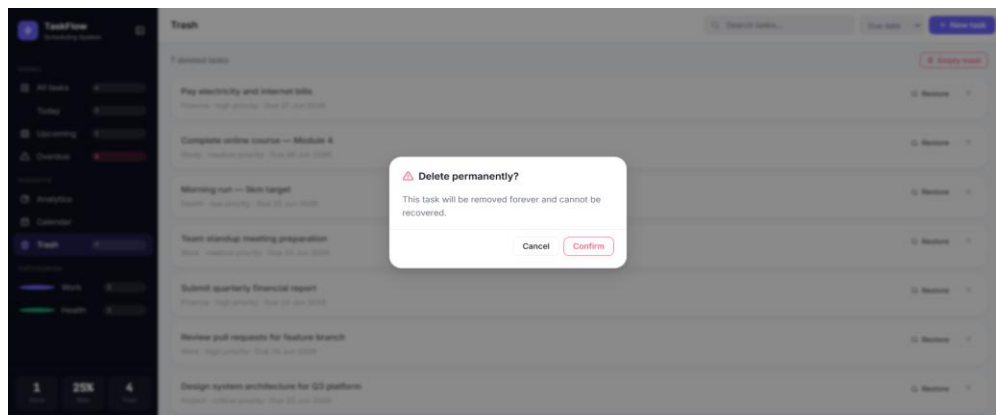


Figure 4.4: Permanent Delete Confirmation Dialog

4.5 Search and Filtering

Figure 4.5 demonstrates the real-time search functionality. As the user types in the search box, the task list updates instantly to show only tasks whose title, notes, category, or tags contain the search string. No button press or form submission is required; the results update on every keystroke. A clear button appears within the search box when text is present, allowing one-click clearing of the search query.

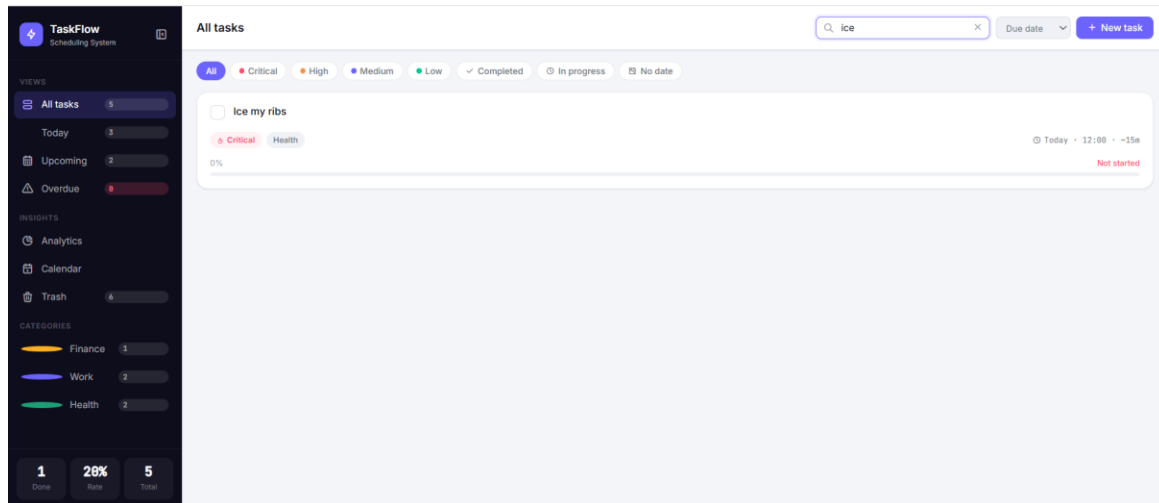


Figure 4.5: Real-Time Search Filtering Tasks

In the example shown, the search string "ice" matches the task "Ice my ribs," which is the only task displayed. In addition to search, the filter chip row provides one-click filtering by priority level (Critical, High, Medium, Low), completion status (Completed), activity status (In progress), and date status (No date). The active chip is highlighted. The sort selector allows sorting by due date, priority, creation date, progress, or name alphabetically. Starred tasks always float to the top of the list regardless of sort order, ensuring the user's most important tasks remain visible.

4.6 Analytics Dashboard

Figure 4.6 shows the Analytics view. The dashboard provides a six-card summary row at the top showing Total tasks, Completed, Completion rate, Overdue, In progress, and Starred counts — each with a distinct icon and colour. Below the summary row, two side-by-side panels display horizontal bar charts for Priority breakdown and By category, showing the distribution of tasks across priority levels and categories respectively.

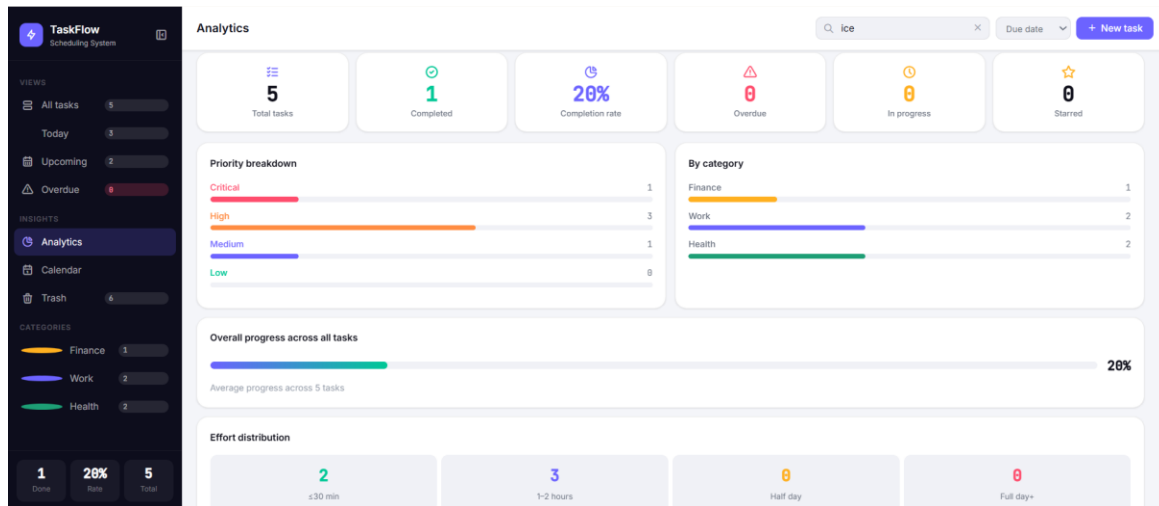


Figure 4.6: Analytics Dashboard

A full-width panel shows the Overall progress across all tasks as a gradient progress bar with the average percentage displayed numerically. Below it, an Effort distribution panel shows the count of tasks in four effort bands: 30 minutes or less, 1–2 hours, half day, and full day+. These statistics give the user a productivity overview without requiring any manual computation or data export. All analytics data is computed in real time from the current task array using pure JavaScript array operations (`filter`, `reduce`, `forEach`), and the analytics view updates immediately when the user navigates to it, always reflecting the most current state of the task data.

CHAPTER FIVE

SUMMARY AND CONCLUSION

5.1 Summary

This project set out to design and implement a computerized task scheduling system that addresses the limitations of existing tools — specifically their dependence on internet connectivity, server infrastructure, and subscription fees — while providing a comprehensive, professional-grade task management experience through a modern web interface. The resulting system, Task Flow, was implemented as a three-file client-side web application using HTML, CSS, and JavaScript. It provides the following features: task creation and editing with a full-featured modal form; priority labelling with four levels displayed as colour-coded badges; progress bars with dynamic colour coding; subtask management with automatic progress computation; rescheduling with a reason note and visible audit badge; soft-deletion with a trash recovery system and permanent delete confirmation dialog; real-time search; sorting by five criteria; filtering by priority, completion status, and date status; a dynamic category navigation system; starred tasks; an analytics dashboard; a monthly calendar view; data persistence via localStorage; keyboard shortcuts; and toast notification feedback for every action.

The Big Bang SDLC model was adopted as the development methodology and proved well-suited to the single-developer, well-scoped nature of the project. A comprehensive use case analysis identified sixteen distinct user interactions, all of which were implemented and verified through functional testing. All stated objectives were achieved: the existing landscape

of task management tools was reviewed; a user-friendly, responsive web interface was designed; all core task management features were implemented; persistent local data storage was implemented using `localStorage`; and the system was evaluated through functional testing on a desktop browser environment.

5.2 Conclusion

The TaskFlow computerized task scheduling system demonstrates that a feature-rich, professional-grade productivity tool can be built entirely with open, universally supported web technologies — HTML, CSS, and JavaScript — without any server infrastructure, external libraries, or subscription services. The system is deployable by simply opening a single HTML file in any modern web browser, making it accessible to users regardless of their internet connectivity or technical background.

The project contributes to the field of web-based productivity software in two ways. First, it provides a practical, usable tool that addresses the genuine scheduling and task management needs of students and professionals in resource-constrained environments. Second, it demonstrates several modern JavaScript development patterns — including event delegation, dynamic DOM generation, `localStorage` integration, and real-time filtering — that serve as instructive reference implementations for students of web development. The analytics dashboard demonstrates that meaningful data visualization is possible using only standard browser APIs and CSS, without requiring charting libraries or canvas-based rendering.

5.3 Recommendation

Based on the experience of developing TaskFlow and the limitations identified during testing, the following recommendations are made for future development:

- **Backend Integration:** A backend server (using technologies such as Node.js with Express, or Python with Flask) and a database (PostgreSQL or MongoDB) would enable multi-device synchronisation, user authentication, and data backup, transforming TaskFlow from a personal tool into a team productivity platform.
- **Multi-User Collaboration:** Building on backend integration, real-time collaborative features — such as task assignment, comment threads, and shared task lists — would make TaskFlow suitable for organisational deployment. WebSocket-based real-time updates would ensure that all collaborators see changes immediately.
- **Mobile Native Application:** A React Native or Flutter mobile application sharing the same backend would extend TaskFlow to mobile devices, providing push notifications for due-date reminders — a feature not possible in a pure browser-based application without service workers.
- **Machine Learning Integration:** Incorporating machine learning models to suggest task priorities, estimate effort, or predict task completion likelihood based on historical data would add an intelligent automation layer, moving the system toward prescriptive scheduling.
- **Accessibility Improvements:** A full accessibility audit using WCAG 2.1 guidelines — including improved keyboard navigation, ARIA roles and labels, and high-contrast mode support — would ensure the system is usable by individuals with disabilities.

- Recurring Task Automation: The current implementation stores a repeat property on tasks but does not automatically generate the next occurrence when a recurring task is completed. Future development should implement automatic recurrence generation on task completion.

REFERENCES

Ahmed, K., Okafor, J., & Musa, R. (2020). Design and implementation of a web-based project management system using Node.js and MongoDB. *International Journal of Computer Science and Information Technology*, 12(4), 115–129. <https://doi.org/10.5121/ijcsit.2020.12409>

Atlassian. (2023). Trello: Manage your team's projects from anywhere. <https://trello.com>

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for agile software development. Agile Alliance. <https://agilemanifesto.org>

Bernstein, M. S., Kairam, S., Suh, B., Hong, L., & Chi, E. H. (2014). A direct manipulation interface for personal task management. *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 32(1), 1–10. <https://doi.org/10.1145/2556288.2557029>

Covey, S. R. (2004). *The 7 habits of highly effective people: Powerful lessons in personal change* (Revised ed.). Free Press.

Doist. (2023). Todoist: The to-do list app to organise your work and life. <https://todoist.com>

ECMA International. (2023). ECMAScript 2023 language specification (14th ed.).
<https://www.ecma-international.org/publications-and-standards/standards/ecma-262/>

Flanagan, D. (2020). JavaScript: The definitive guide (7th ed.). O'Reilly Media.

Frisbie, M. (2019). Professional JavaScript for web developers (4th ed.). Wiley.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The unified software development process. Addison-Wesley.

Khalid, M. N., Iqbal, M., Rasheed, K., & Abid, M. M. (2020). Web-based task scheduling and priority management tool using PHP and MySQL. International Journal of Advanced Computer Science and Applications, 11(3), 78–86.
<https://doi.org/10.14569/IJACSA.2020.0110311>

Markus, M. L., & Mao, J. Y. (2004). Participation in development and implementation: Updating an old, tired concept for today's IS contexts. Journal of the Association for Information Systems, 5(11), 514–544. <https://doi.org/10.17705/1jais.00062>

MDN Web Docs. (2023). CSS: Cascading style sheets. Mozilla Developer Network.
<https://developer.mozilla.org/en-US/docs/Web/CSS>

Microsoft. (2023). Microsoft To Do: Lists, tasks, and reminders. <https://to-do.microsoft.com>

Norman, D. A. (2013). *The design of everyday things* (Revised and expanded ed.). Basic Books.

Notion Labs. (2023). Notion: The all-in-one workspace for your notes, tasks, wikis, and databases. <https://www.notion.so>

Okafor, C., & Eze, T. (2022). localStorage as a persistence layer for offline web applications: A performance evaluation. *Nigerian Journal of Technology*, 41(2), 302–310. <https://doi.org/10.4314/njt.v41i2.12>

Pinedo, M. L. (2016). *Scheduling: Theory, algorithms, and systems* (5th ed.). Springer. <https://doi.org/10.1007/978-3-319-26580-3>

Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26(8), 1–9.

Salami, A. O., & Okon, E. E. (2021). Assessment of digital task management tools among academic staff of Nigerian universities. *Journal of Educational Technology and Society*, 24(3), 45–57.

Tutorialspoint. (2023). SDLC Big Bang model. Tutorialspoint. https://www.tutorialspoint.com/sdlc/sdlc_big_bang_model.htm

W3C. (2014). HTML5: A vocabulary and associated APIs for HTML and XHTML. World Wide Web Consortium. <https://www.w3.org/TR/html5/>